

CAPÍTULO 8

MONITOREO Y GESTIÓN DE PROCESOS LINUX

JAVIER LLUESMA APARICI 2ASIX
SISTEMAS OPERATIVOS EN RED
MEMORIA RED HAT



ÍNDICE

1. MONITOREO Y GESTIÓN DE PROCESOS EN LINUX	3
PROCESO, ENTORNO DE PROCESO Y CREACIÓN DE PROCESOS	3
ESTADOS DEL PROCESO EN LINUX	3
IMPORTANCIA DE LOS ESTADOS	4
COMANDOS PARA LISTAR PROCESOS	4
OPCIONES:	4
EJEMPLOS DE USO:	5
SEÑALES	5
NOTAS ADICIONALES	5
2. CONTROL DE TRABAJOS EN BASH	5
OBJETIVOS	5
DESCRIPCIÓN DE TRABAJOS Y SESIONES	6
PRIMER PLANO VS. SEGUNDO PLANO	6
SESIONES	6
COMANDOS CLAVE	6
EJECUCIÓN DE TRABAJOS EN SEGUNDO PLANO	6
MANEJO DE TRABAJOS	6
INFORMACIÓN DE PROCESOS	7
NOTAS IMPORTANTES	7
3. EJERCICIO GUIADO: CONTROL DE TRABAJOS	7
4. FINALIZACIÓN DE PROCESOS	11
CONTROL DE PROCESOS CON SEÑALES	11
ENVÍO DE SEÑALES	11
COMANDOS ÚTILES	11
CONTROL DE TRABAJOS	12
CIERRE DE SESIÓN DE USUARIOS	12
RECOMENDACIÓN PARA FINALIZAR PROCESOS	12
5. EJERCICIO GUIADO: FINALIZACIÓN DE PROCESOS	12
6. MONITOREO DE ACTIVIDAD DE PROCESOS	14
PROMEDIO DE CARGA	14
CÁLCULO:	15
INTERPRETACIÓN DE VALORES:	15

EJEMPLO:	15
CARGA POR CPU:	15
COMANDO LSCPU	15
COMANDO TOP	15
COLUMNAS IMPORTANTES:	15
TECLAS ESENCIALES EN TOP:	16
7. EJERCICIO GUIADO: MONITOREO DE ACTIVIDAD DE PROCESOS	16
8. TRABAJO DE LABORATORIO: MONITOREO Y GESTIÓN DE PROCESOS LINUX	19

1. MONITOREO Y GESTIÓN DE PROCESOS EN LINUX

PROCESO, ENTORNO DE PROCESO Y CREACIÓN DE PROCESOS

PROCESO --> Instancia de un programa ejecutable en funcionamiento.

Consta de:

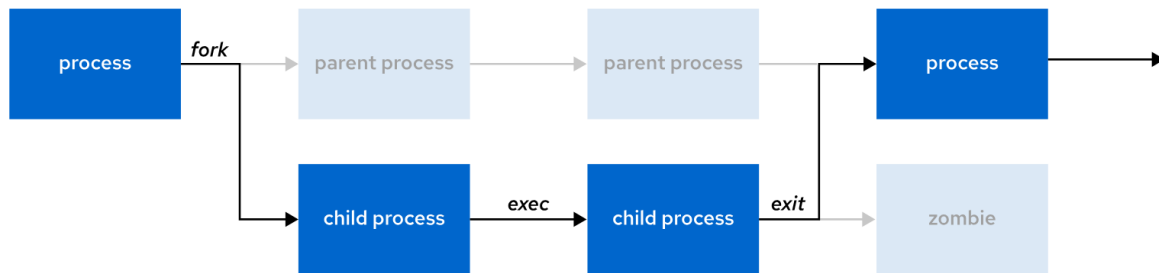
- Espacio de direcciones (memoria asignada).
- Propiedades de seguridad (credenciales y privilegios).
- Uno o más subprocesos de ejecución.
- Estado del proceso

ENTORNO DE PROCESO --> Lista de información que incluye:

- Variables locales y globales.
- Contexto de programación actual.
- Recursos del sistema, (descriptores de archivos, puertos de red).

CREACIÓN DE PROCESOS:

- Proceso principal puede crear procesos secundarios (bifurcación)
- Herencia de identidades de seguridad y recursos.
- Cada proceso nuevo recibe un PID único.



ESTADOS DEL PROCESO EN LINUX

En Ejecución (R) --> El proceso está ejecutándose en una CPU o está listo para ejecutarse. Puede estar ejecutando rutinas de usuario o del kernel.

En Espera (S) --> El proceso espera una condición específica (hardware, recurso, señal) para volver a estar en ejecución.

Subestado --> K (TASK_KILLABLE):

-Similar a S, pero puede responder a señales de anulación.

En Espera Ininterrumpible (D) --> El proceso está en espera y no responde a señales. Se utiliza para evitar estados imprevistos en dispositivos.

En Espera Idle (I) --> Subconjunto de D. No se cuenta en el promedio de carga del kernel. Acepta señales fatales.

Detenido (T) --> El proceso ha sido detenido por otra señal. Puede reanudarse y volver a estar en ejecución.

Depurado (T) --> Proceso detenido temporalmente para depuración, comparte el mismo indicador que T.

Zombie (Z) --> Proceso que ha terminado, pero aún tiene un PID asignado. Espera que su proceso principal lo limpie.

Muerto (X) --> Proceso completamente liberado después de que el proceso principal limpia su estructura. No es observable en listas de procesos.

IMPORTANCIA DE LOS ESTADOS

- Comprender la comunicación entre el kernel y los procesos.
- Los estados se muestran en comandos como top y ps.

COMANDOS PARA LISTAR PROCESOS

- **ps**: Muestra información detallada de procesos actuales.

OPCIONES:

- **UNIX (POSIX):** -
- **BSD:** sin guion
- **GNU:** --

EJEMPLOS DE USO:

- ps aux: Muestra todos los procesos.
- ps -ef: Muestra todos los procesos con más detalles.
- ps --forest: Muestra procesos en formato de árbol.

Monitor Active Processes (ps)	
ps	returns PIDs of processes running under your account
a or -a	lists all processes running on a terminal
e	returns details of shell environment
f	returns output as a tree structure
l or -l	returns more detailed listing of processes
u	returns username and start time form processes
x	lists processes without associated terminal

SEÑALES

Usadas para suspender, detener, reanudar, finalizar o interrumpir procesos.

NOTAS ADICIONALES

Los procesos zombies se etiquetan como "exiting" o "defunct".
La salida de ps se ordena por PID, no cronológicamente.

2. CONTROL DE TRABAJOS EN BASH

OBJETIVOS

Gestionar múltiples procesos desde una sola sesión de terminal.

DESCRIPCIÓN DE TRABAJOS Y SESIONES

Trabajo: Asociado a cada tubería ingresada en el prompt de shell.
Grupo de Procesos: Todos los procesos en una tubería son parte del mismo trabajo.

PRIMER PLANO VS. SEGUNDO PLANO

Primer Plano: Solo un trabajo puede leer entradas del teclado.

Segundo Plano: Procesos que no pueden leer del teclado, pero pueden escribir en el terminal.

SESIONES

Cada terminal es su propia sesión, con un proceso en primer plano y múltiples en segundo plano.

COMANDOS CLAVE

- **ps**: Muestra información sobre los procesos y el terminal de control.
- **jobs**: Lista los trabajos en la sesión actual.
- **fg**: Trae un trabajo en segundo plano al primer plano.
- **bg**: Reanuda un trabajo suspendido en segundo plano.

EJECUCIÓN DE TRABAJOS EN SEGUNDO PLANO

- Para iniciar un comando en segundo plano, añada & al final del comando.
- Ejemplo: sleep 10000 &
- La shell muestra el número de trabajo y el PID del proceso.

MANEJO DE TRABAJOS

- Mostrar trabajos: jobs
- Traer al primer plano: fg %n (donde n es el número del trabajo).
- Suspender un trabajo en primer plano: Presiona Ctrl + Z.
- Reanudar un trabajo suspendido: bg %n.

INFORMACIÓN DE PROCESOS

- ps j: Muestra información detallada sobre los trabajos.
- PID: ID del proceso.
- PPID: ID del proceso padre.
- PGID: ID del líder del grupo de procesos.
- SID: ID del líder de sesión.

NOTAS IMPORTANTES

- La shell advierte si intentas salir con trabajos suspendidos; estos se anulan si intentas salir de inmediato.
- El signo + indica el trabajo predeterminado actual; el signo - indica el trabajo anterior.

3. EJERCICIO GUIADO: CONTROL DE TRABAJOS

- En primer lugar, accederemos al laboratorio, hacemos ssh a servera y creamos el siguiente directorio, luego el script y damos permisos.

```
student@servera:~  
[student@workstation ~]$ lab start process-control  
Script process-control not in course library rhcsa-classroom  
[student@workstation ~]$ lab start processes-control  
  
Starting lab.  
  
· Checking lab systems ..... SUCCESS  
  
[student@workstation ~]$ ssh student@servera  
Warning: Permanently added 'servera' (ED25519) to the list of known hosts.  
Activate the web console with: systemctl enable --now cockpit.socket  
  
Register this system with Red Hat Insights: insights-client --register  
Create an account or view all your systems at https://red.ht/insights-dashboard  
[student@servera ~]$ mkdir /home/student/bin  
[student@servera ~]$ nano /home/student/bin/control  
[student@servera ~]$ chmod +x /home/student/bin/control  
[student@servera ~]$
```

-Ejecutamos el script y verificamos en el segundo terminal que el nuevo proceso se está escribiendo.

```
[student@servera ~]$ control technical
```

-Visualización terminal derecho.

```
[student@servera ~]$ tail -f ~/control outfile  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical  
technical technical technical technical technical technical technical technical
```

-A continuación, suspendemos el proceso y comprobamos que se ha suspendido.

```
[student@servera ~]$ control technical  
^Z  
[1]+  ⏏ Stopped                  control technical  
[student@servera ~]$
```

-Visualización terminal derecho.


```
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical
```

-Ahora visualizamos la lista de trabajos y reiniciamos el trabajo control en segundo plano.

```
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
[student@servera ~]$ bg
[1]+  control technical &
[student@servera ~]$
```

-De nuevo verificamos que vuelve a estar en funcionamiento y confirmamos que está produciendo salida de nuevo.

```
[student@servera ~]$ jobs
[1]+  Running                  control technical &
[student@servera ~]$
```

-Visualización terminal derecho.

```
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
technical technical technical technical technical technical technical technical
```

-Iniciamos dos nuevos procesos en segundo plano y comprobamos que están en ejecución y adjuntándose al archivo.

```

[student@servera ~]$ control documents &
[2] 1663
[student@servera ~]$ control database &
[3] 1680
[student@servera ~]$ jobs
[1]  Running          control technical &
[2]- Running         control documents &
[3]+ Running         control database &
[student@servera ~]$

```

-Visualización terminal derecho.

```

abase documents technical database documents technical database documents techni
cal database documents technical database documents technical database documents
technical database documents technical database documents technical database do
cuments technical database documents technical database documents technical data
base documents technical database documents technical database documents technic
al database documents technical database documents technical database documents
technical database

```

-Ponemos en primer plano el proceso control technical y suspendemos el proceso, de nuevo verificamos que está suspendido y que ya no está enviando salida.

```

[student@servera ~]$ fg %1
control technical
^Z
[1]+  Stopped          control technical
[student@servera ~]$ jobs
[1]+  Stopped          control technical
[2]  Running          control documents &
[3]- Running          control database &
[student@servera ~]$

```

-Visualización terminal derecho.

```

e documents database documents database documents database documents database do
cumen]s database documents database documents database documents database docume
nts database documents database documents database documents database documents
database documents database documents database documents database documents data
base documents database documents database documents database documents database
documents database documents database documents database documents database doc
uments database

```

-Realizamos los mismos pasos con control documents.

```

[student@servera ~]$ fg %2
control documents
^C
[student@servera ~]$ jobs
[1]+  Stopped          control technical
[3]-  Running          control database &
[student@servera ~]$

```

```
database database database database database database database database datab
base database database database database database database database database dat
base database database database database database database database database dat
abase database database database database database database database database da
tabase database database database database database database database database d
atabase database database database database database database database database
database database
```

```
[student@servera ~]$ ps jT
```

PPID	PID	PGID	SID	TTY	TPGID	STAT	UID	TIME	COMMAND
1463	1464	1464	1464	pts/1	4611	Ss	1000	0:00	-bash
1464	1481	1481	1464	pts/1	4611	T	1000	0:00	/bin/bash /ho
1464	1680	1680	1464	pts/1	4611	S	1000	0:00	/bin/bash /ho
1481	2254	1481	1464	pts/1	4611	T	1000	0:00	sleep 1
1680	4610	1680	1464	pts/1	4611	S	1000	0:00	sleep 1
1464	4611	4611	1464	pts/1	4611	R+	1000	0:00	ps jT

```
[student@servera ~]$
```

```
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
[3]-  Running                  control database &
[student@servera ~]$ fg %3
control database
^C
[student@servera ~]$ jobs
[1]+  Stopped                  control technical
[student@servera ~]$
```

[illegible]

Las señales son interrupciones de software enviadas a procesos para indicar eventos o errores. Estas son las más comunes:

- **SIGHUP (1):** Reinicia un proceso sin cerrarlo.
- **SIGINT (2):** Interrumpe el proceso (Ctrl + C).
- **SIGQUIT (3):** Interrumpe el proceso con volcado (Ctrl + \).
- **SIGKILL (9):** Mata el proceso inmediatamente (no puede ignorarse).
- **SIGTERM (15):** Finaliza un proceso de forma segura.
- **SIGSTOP (19):** Suspende el proceso (Ctrl + Z para detenerlo).
- **SIGCONT (18):** Reanuda un proceso detenido.

ENVÍO DE SEÑALES

-El comando kill envía señales a procesos identificados por su PID.

```
kill -SIGTERM 1234 # Envía SIGTERM al proceso con PID 1234
kill -9 1234       # Mata (SIGKILL) el proceso 1234
```

-Usa ps aux para listar los procesos y grep para filtrar por nombre.

COMANDOS ÚTILES

- **pkill:** Mata procesos según criterios como el nombre del proceso o usuario.

```
pkill -SIGKILL -u usuario # Mata todos los procesos de un usuario
```

- **pgrep:** Lista los procesos sin matarlos, útil para identificar procesos.

```
pgrep -l -u usuario # Lista los procesos de un usuario
```

- **killall:** Termina todos los procesos que coincidan con el nombre.

```
killall nombre_proceso # Mata todos los procesos con ese nombre
```

CONTROL DE TRABAJOS

Para procesos en segundo plano:

- Usamos jobs para listar los trabajos en segundo plano.
- Finalizar un trabajo específico con kill %N, donde N es el número del trabajo:

```
kill %1 # Termina el trabajo 1
```

CIERRE DE SESIÓN DE USUARIOS

- w para ver sesiones activas y recursos usados.
- Finalizar sesiones con `pkill -t ttyN` donde `ttyN` es la terminal.

RECOMENDACIÓN PARA FINALIZAR PROCESOS

- Primero intenta con **SIGTERM** para una finalización limpia.
- Si falla, usa **SIGINT** o **SIGKILL** como último recurso.

5. EJERCICIO GUIADO: FINALIZACIÓN DE PROCESOS

-Accedemos a la workstation y luego de realizar el ssh creamos el nuevo directorio y script que ejecutaremos.

```
[student@servera ~]$ mkdir /home/student/bin  
[student@servera ~]$ cd /home/student/bin  
[student@servera bin]$ nano /home/student/bin/instance  
[student@servera bin]$ chmod +x /home/student/bin/instance
```

-Lo hacemos ejecutable e iniciamos 3 procesos en segundo plano.

```
[student@servera bin]$ instance network &  
[1] 1110  
[student@servera bin]$ instance interface &  
[2] 1113  
[student@servera bin]$ instance connection &  
[3] 1117
```

-Desde otro terminal verificamos que los 3 procesos se adjuntan correctamente al archivo.

```
[student@servera ~]$ tail -f ~/instance_outfile  
networknetworkinterfaceconnectionnetworkinterfaceconnectionnetwo  
rkinterfaceconnectionnetworkinterfaceconnectionnetworkinterfaceconnectionnetwork
```

-Enumeramos los trabajos.

```
[student@servera bin]$ jobs
[1]    Running                  instance network &
[2]-   Running                  instance interface &
[3]+   Running                  instance connection &
```

-Detenemos el proceso instance network y verificamos que está parado.

```
[student@servera bin]$ kill -SIGSTOP %1
[student@servera bin]$ jobs
[1]+   Stopped                  instance network
[2]    Running                  instance interface &
[3]-   Running                  instance connection &
```

-Comprobamos en el otro terminal que la palabra network ya no está anexada al archivo.

```
nectioninterfaceconnectioninterfaceconnectioninterfaceconnectioninterfaceconnect
ioninterfaceconnection
```

-Finalizamos el proceso interface y realizamos las mismas comprobaciones realizadas con el proceso anterior.

```
[student@servera bin]$ kill -SIGTERM %2
[2]    Terminated              instance interface
[student@servera bin]$ jobs
[1]+   Stopped                  instance network
[3]-   Running                  instance connection &
[student@servera bin]$
```

-Visualización terminal derecho.

```
onconnectionconnectionconnectionconnectionconnectionconnectionconnectionconnecti
onconnectionconnectionconnectionconnectionconnectionconnectionconnectionconnecti
on
```

-Reanudamos el proceso instance network y de nuevo comprobamos.

```
[student@servera bin]$ kill -SIGCONT %1
[student@servera bin]$ jobs
[1]+   Running                  instance network &
[3]-   Running                  instance connection &
[student@servera bin]$
```

-Visualización terminal derecho.


```
onconnectionconnectionconnectionconnectionconnectionconnectionnetworkconnectionn
etworkconnectionnetworkconnectionnetworkconnectionnetworkconnectionnetworkconnec
tionnetworkconnectionnetworkconnectionnetwork
```

-Finalizamos el proceso instance network y instance connection y comprobamos.

```
student@servera bin]$ kill -SIGTERM %1
student@servera bin]$ kill -SIGTERM %3
1)+ Terminated          instance network
student@servera bin]$ jobs
3)+ Terminated          instance connection
```

-Enumeramos los procesos en ejecución actuales

```
[student@servera bin]$ ps -ef | grep tail
student    1245    1185  0 05:40 pts/1    00:00:00 tail -f /home/student/instan
ce_outfile
student    1703    1070  0 06:05 pts/0    00:00:00 grep --color=auto tail
[student@servera bin]$
```

-Finalizamos el proceso tail y verificamos que los procesos ya no están ejecutándose.

```
[student@servera bin]$ pkill -SIGTERM tail
[student@servera bin]$ ps -ef | grep tail
student    1708    1070  0 06:08 pts/0    00:00:00 grep --color=auto tail
[student@servera bin]$
```

-Visualización terminal derecho.

```
ionnetworkconnectionnetworkconnectionnetworkconnectionnetworkconnectionnetworkco
nnectionTerminated
[student@servera ~]$
```

6. MONITOREO DE ACTIVIDAD DE PROCESOS

PROMEDIO DE CARGA

-El promedio de carga mide la carga percibida del sistema en períodos de 1, 5 y 15 minutos, indicando cuántos procesos están esperando recursos (CPU, disco, red).

CÁLCULO:

- **Estados considerados:**
 - Procesos en ejecución (R).
 - Procesos en espera de E/S (D).
- Incluye uso de CPU, disco y red.

INTERPRETACIÓN DE VALORES:

- **< 1:** Uso eficiente de recursos.

- > 1: Saturación de recursos, con posibles retrasos.

EJEMPLO:

```
$ uptime
15:29:03 up 14 min, 2 users, load average: 2.92, 4.48, 5.20
```

- Valores corresponden a los últimos 1, 5 y 15 minutos.

CARGA POR CPU:

Si un sistema tiene 4 CPU lógicas, divida los promedios entre 4:

```
load average per CPU: 0.73, 1.12, 1.30
```

- El sistema estuvo saturado un 12% en los últimos 5 minutos.

COMANDO LSCPU

- Muestra el número de CPU lógicas del sistema.

Ejemplo:

```
$ lscpu
CPU(s): 4
```

COMANDO TOP

Vista dinámica en tiempo real de los procesos del sistema.

COLUMNAS IMPORTANTES:

- **PID:** ID del proceso.
- **USER:** Usuario propietario del proceso.
- **VIRT:** Memoria virtual usada.
- **RES:** Memoria física usada.
- **S:** Estado del proceso:
 - **R:** Ejecutable.
 - **D:** Esperando E/S.
 - **S:** En espera.
 - **Z:** Zombie.
- **TIME:** Tiempo total de CPU utilizado.

- **COMMAND:** Nombre del comando.

TECLAS ESENCIALES EN TOP:

TECLA	FUNCIÓN
? o h	Ayuda Interactiva
1	Mostrar CPU individuales
Shift + p	Ordenar por uso de CPU
Shift + m	Ordenar por uso de memoria
u o Shift + u	Filtrar por nombre de usuario
k	Eliminar un proceso (PID)
r	Cambiar prioridad de un proceso
q	Salir

7. EJERCICIO GUIADO: MONITOREO DE ACTIVIDAD DE PROCESOS

-Accedemos a la workstation y luego de realizar el ssh creamos el nuevo directorio y script que ejecutaremos y lo hacemos ejecutable.

```
[student@servera ~]$ mkdir /home/student/bin
mkdir: cannot create directory '/home/student/bin': File exists
[student@servera ~]$ nano /home/student/bin/monitor
[student@servera ~]$ chmod a+x /home/student/bin/monitor
[student@servera ~]$
```

-Ejecutamos el comando top y vemos todos los procesos.

```
top - 07:03:13 up 14 min, 2 users, load average: 0.01, 0.02, 0.01
Tasks: 122 total, 1 running, 121 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 0.2 sy, 0.0 ni, 99.8 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1774.8 total, 1444.8 free, 188.2 used, 141.7 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 1437.7 avail Mem

  PID USER      PR  NI  VIRT  RES  SHR S %CPU  %MEM    TIME+  COMMAND
 1250 student   20   0 225944 4340 3628 R  0.7   0.2   0:00.16 top
    1 root      20   0 105932 16984 10336 S  0.0   0.9   0:01.64 systemd
    2 root      20   0      0      0      0 S  0.0   0.0   0:00.00 kthreadd
    3 root      0 -20      0      0      0 I  0.0   0.0   0:00.00 rcu_gp
    4 root      0 -20      0      0      0 I  0.0   0.0   0:00.00 rcu_par+
    6 root      0 -20      0      0      0 I  0.0   0.0   0:00.00 kworker+
    9 root      0 -20      0      0      0 I  0.0   0.0   0:00.00 mm_perc+
   10 root      20   0      0      0      0 S  0.0   0.0   0:00.00 rcu_tas+
   11 root      20   0      0      0      0 S  0.0   0.0   0:00.00 rcu_tas+
   12 root      20   0      0      0      0 S  0.0   0.0   0:00.00 rcu_tas+
   13 root      20   0      0      0      0 S  0.0   0.0   0:00.00 ksoftir+
   14 root      20   0      0      0      0 I  0.0   0.0   0:00.03 rcu_pre+
   15 root      rt    0      0      0      0 S  0.0   0.0   0:00.00 migrati+
```

-Determinamos cantidad de CPUs lógicas que tiene la máquina con lscpu.

```
[student@servera ~]$ lscpu
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         46 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                2
On-line CPU(s) list:   0,1
Vendor ID:             GenuineIntel
Model name:            Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
CPU family:            6
```

-Ejecutamos una sola instancia del archivo ejecutable monitor.

```
[student@servera ~]$ monitor &
[1] 1260
```

-En el segundo terminal monitoreando el comando top pulsamos l, t y m individualmente para alternar carga, subprocessos y líneas de encabezado de memoria.

```
Tasks: 119 total,  1 running, 118 sleeping,  0 stopped,  0 zombie
%Cpu(s):  6.6/0.8   7[||||]
MiB Mem : 18.8/1774.8  [|||||||]
MiB Swap:  0.0/0.0   [

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1260	student	20	0	222652	3712	3256	S	13.6	0.2	2:10.35	monitor
1250	student	20	0	225944	4340	3628	R	0.3	0.2	0:01.35	top
1	root	20	0	105932	16984	10336	S	0.0	0.9	0:01.65	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par+
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker+
9	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_perc+
10	root	20	0	0	0	0	S	0.0	0.0	0:00.00	rcu_tas+

-ID de proceso (PID) para el proceso monitor y porcentaje de CPU para el proceso, que se espera que sea alrededor del 15 % o el 25 %.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1260	student	20	0	222652	3712	3256	S	20.9	0.2	2:24.10	monitor

-Ejecutamos una segunda y tercera instancia y observamos el ID y que el porcentaje de CPU sea entre 15 y 25%.

```
[student@servera ~]$ monitor &
[2] 2491
[student@servera ~]$ monitor &
[3] 2504
[student@servera ~]$
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2504	student	20	0	222652	3860	3408	S	20.6	0.2	0:03.07	monitor
1260	student	20	0	222652	3712	3256	S	20.3	0.2	4:23.74	monitor
2491	student	20	0	222652	3932	3480	S	13.3	0.2	0:04.10	monitor

-Observamos como el promedio de carga sigo siendo menor a 1.

```
top - 07:31:56 up 43 min,  2 users,  load average: 0.86, 0.36, 0.19
```

-Añadimos más instancias y comprobamos como ahora sí el promedio de carga es mayor a 1.

```
top - 07:34:00 up 45 min,  2 users,  load average: 2.64, 1.01, 0.44
Tasks: 139 total,  2 running, 137 sleeping,  0 stopped,  0 zombie
%Cpu(s): 60.6 us,  5.8 sy,  0.0 ni, 33.1 id,  0.0 wa,  0.3 hi,  0.2 si,  0.0 st
MiB Mem : 1774.8 total, 1414.9 free, 193.1 used, 166.9 buff/cache
MiB Swap:  0.0 total,  0.0 free,  0.0 used. 1429.7 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1260	student	20	0	222652	3712	3256	S	14.3	0.2	4:59.96	monitor
2881	student	20	0	222652	3912	3456	S	13.6	0.2	0:12.14	monitor
2900	student	20	0	222652	3868	3420	S	13.0	0.2	0:11.08	monitor
2883	student	20	0	222652	3948	3492	S	12.6	0.2	0:12.01	monitor
2491	student	20	0	222652	3936	3480	S	11.6	0.2	0:39.58	monitor
2504	student	20	0	222652	3864	3408	S	11.6	0.2	0:38.87	monitor
3090	student	20	0	222652	3868	3416	S	11.6	0.2	0:04.47	monitor
3102	student	20	0	222652	3688	3236	S	11.6	0.2	0:04.48	monitor
3081	student	20	0	222652	3836	3384	S	11.3	0.2	0:04.83	monitor
3096	student	20	0	222652	3928	3476	S	11.0	0.2	0:04.44	monitor
3076	student	20	0	222652	3864	3412	R	10.6	0.2	0:04.95	monitor

-A continuación, para acabar con todos los procesos uno a uno vamos al terminal con top abierto y pulsamos k a continuación se observa el prompt que está justo debajo, por lo que pulsamos Enter y el proceso se finaliza.

```
PID to signal/kill [default pid = 3081]
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3081	student	20	0	222652	3840	3384	R	14.0	0.2	0:19.60	monitor

```
Send pid 1260 signal [15/sigterm]
```

-Hasta acabar con todos.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2883	student	20	0	222652	3948	3492	S	21.3	0.2	0:47.32	monitor
3076	student	20	0	222652	3868	3412	S	21.3	0.2	0:40.91	monitor
2504	student	20	0	222652	3864	3408	S	13.0	0.2	1:14.40	monitor
1	root	20	0	105932	16984	10336	S	0.0	0.9	0:01.67	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	0	-20	0	0	0	T	0.0	0.0	0:00.00	rcu_gp

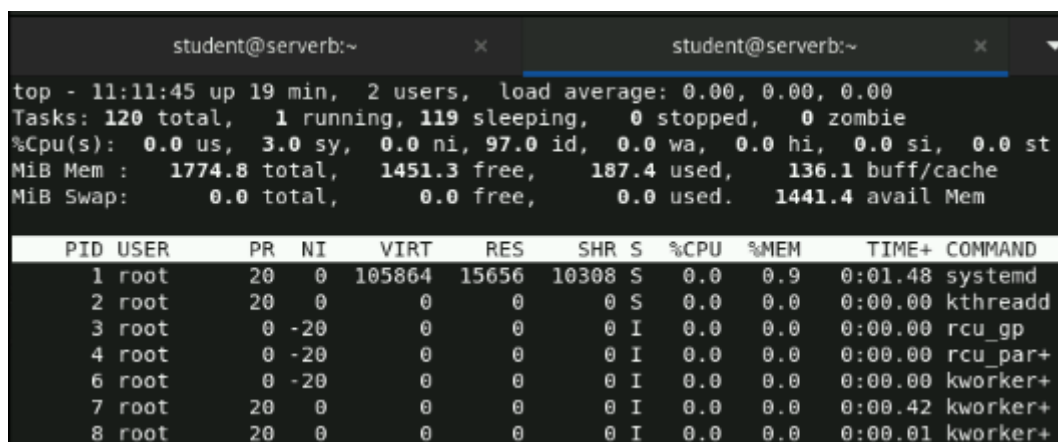
8. TRABAJO DE LABORATORIO: MONITOREO Y GESTIÓN DE PROCESOS LINUX

1. En workstation, abra dos ventanas de terminal una al lado de la otra. En esta sección, estas terminales se denominan izquierda y derecha. En cada ventana de terminal, inicie sesión en serverb con el usuario student.

Cree el script process101 en el directorio /home/student/bin. El script process101 genera una carga de CPU artificial.

```
[student@serverb ~]$ mkdir /home/student/bin
[student@serverb ~]$ nano /home/student/bin/process101
[student@serverb ~]$ chmod +x /home/student/bin/process101
```

2. En la shell de terminal derecha, ejecute la utilidad top.



student@serverb:~ x student@serverb:~ x

top - 11:11:45 up 19 min, 2 users, load average: 0.00, 0.00, 0.00
Tasks: 120 total, 1 running, 119 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.0 us, 3.0 sy, 0.0 ni, 97.0 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1774.8 total, 1451.3 free, 187.4 used, 136.1 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 1441.4 avail Mem

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	root	20	0	105864	15656	10308	S	0.0	0.9	0:01.48	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par+
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker+
7	root	20	0	0	0	0	I	0.0	0.0	0:00.42	kworker+
8	root	20	0	0	0	0	I	0.0	0.0	0:00.01	kworker+

3. En la shell de terminal izquierda, determine la cantidad de CPU lógicas de la máquina virtual. Ejecute el script process101 en segundo plano.

-2 CPUS lógicas

```
[student@serverb ~]$ lscpu
Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 46 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 2
On-line CPU(s) list: 0-1
```

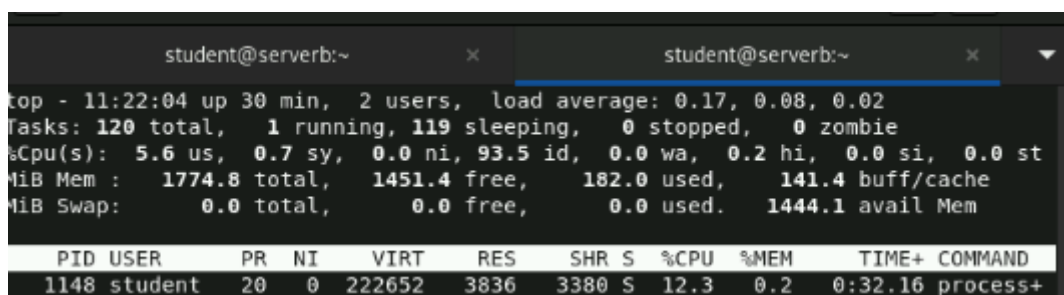
-Ejecutamos el script en segundo plano.

```
[student@serverb ~]$ process101 &
[1] 1148
[student@serverb ~]$
```

4. En la shell de terminal derecha, observe la pantalla de top. Observe el ID del proceso (PID) y vea el porcentaje de CPU que usa el proceso process101. El porcentaje de CPU que usa el proceso debe oscilar entre el 10 % y el 15 %. Alterne la visualización de la utilidad top entre cargas, subprocesos y memoria. Regrese a la pantalla de uso de CPU de la utilidad top.

-Para alternar pulsamos l, t y m individualmente.

-Uso de un 12.3% de CPU y uso alterno.



```
top - 11:22:04 up 30 min, 2 users, load average: 0.17, 0.08, 0.02
Tasks: 120 total, 1 running, 119 sleeping, 0 stopped, 0 zombie
%Cpu(s): 5.6 us, 0.7 sy, 0.0 ni, 93.5 id, 0.0 wa, 0.2 hi, 0.0 si, 0.0 st
MiB Mem : 1774.8 total, 1451.4 free, 182.0 used, 141.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 1444.1 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1148	student	20	0	222652	3836	3380	S	12.3	0.2	0:32.16	process+

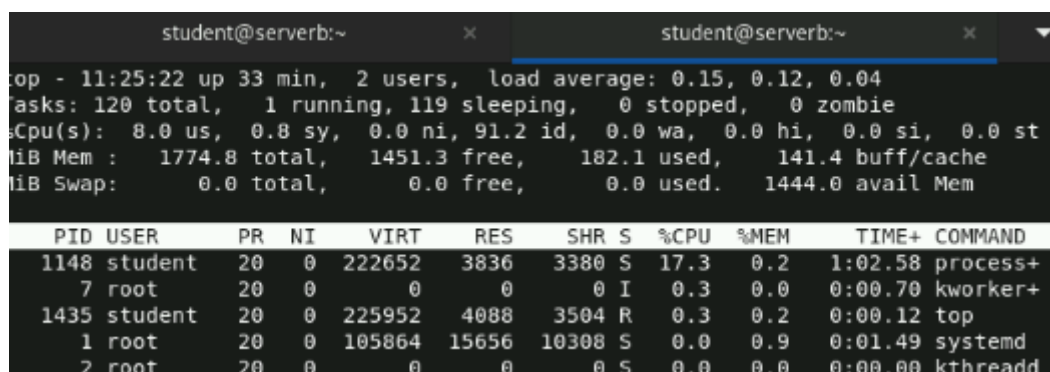
5. Desactive el uso de negrita en la pantalla. Guarde esta configuración para volver a usarla cuando se reinicie top. Confirme que los cambios están guardados.

-Shift + b para quitar negrita y seguido Shift + W para guardar los cambios.



```
Wrote configuration to '/home/student/.config/procps/toprc'
```

-Reiniciamos top y comprobamos que ya no se ve con negrita.



```
top - 11:25:22 up 33 min, 2 users, load average: 0.15, 0.12, 0.04
Tasks: 120 total, 1 running, 119 sleeping, 0 stopped, 0 zombie
%Cpu(s): 8.0 us, 0.8 sy, 0.0 ni, 91.2 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
MiB Mem : 1774.8 total, 1451.3 free, 182.1 used, 141.4 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used, 1444.0 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1148	student	20	0	222652	3836	3380	S	17.3	0.2	1:02.58	process+
7	root	20	0	0	0	0	I	0.3	0.0	0:00.70	kworker+
1435	student	20	0	225952	4088	3504	R	0.3	0.2	0:00.12	top
1	root	20	0	105864	15656	10308	S	0.0	0.9	0:01.49	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.00	kthreadd

6. Copie el script process101 en un nuevo archivo process102 y aumente la carga de CPU artificial a cien mil en el nuevo script. Inicie el proceso process102 en segundo plano.

-Copiamos el archivo y aumentamos la carga de CPU.

```
[student@serverb ~]$ cp /home/student/bin/process101 /home/student/bin/process102
```

```
GNU nano 5.6.1 /home/student/bin/process102
#!/bin/bash

while true; do

    var=1

    while [[ var -lt 1000000 ]]; do

        var=$((var+1))

    done

    sleep 1

done
```

-Damos permisos e iniciamos el proceso en segundo plano.

```
[student@serverb ~]$ chmod +x /home/student/bin/process102
```

```
[student@serverb ~]$ process102 &
[2] 1638
```

7. En la shell de terminal derecha, verifique que el proceso se está ejecutando y que usa la mayoría de los recursos de la CPU. La carga debe ser del 25 % al 35 %.

-Visualización de uso de carga en torno a un 30.9% de la CPU.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1638	student	20	0	222652	3888	3436	S	30.9	0.2	0:20.16	process+

8. El promedio de carga es inferior a 1. Copie process101 a un nuevo script denominado process103. Incremente el recuento de sumas a ochocientos mil. Inicie process103 en segundo plano. Confirme que el promedio de carga sea superior a 1. El promedio de carga puede tardar unos minutos en cambiar.

-Creación e incremento y cambios en el script, damos permisos también.

```
[student@serverb ~]$ cp /home/student/bin/process101 /home/student/bin/process103
```

```

GNU nano 5.6.1 /home/student/bin/process103
#!/bin/bash

while true; do

    var=1

    while [[ var -lt 800000 ]]; do

        var=$((var+1))

    done

    sleep 1

done

[student@serverb ~]$ chmod +x /home/student/bin/process103

```

-Proceso iniciado en segundo plano.

```

[student@serverb ~]$ process103 &
[3] 2364

```

-Promedio de carga superior a 1.

```

student@serverb:~
top - 11:37:08 up 45 min,  2 users,  load average: 1.02, 0.57, 0.29

```

9. En la shell de terminal izquierda, cambie al usuario root. Suspenda el proceso process101. Enumere los trabajos restantes. Observe que el estado del proceso process101 ahora es T.

-Accedemos como root.

```

[student@serverb ~]$ su -
Password:
Last login: Thu Oct  3 11:44:01 EDT 2024 on pts/0

```

-Suspendemos el proceso.

```

[root@serverb ~]# pkill -SIGSTOP process101

```

-Enumeramos los trabajos restantes y observamos el estado del proceso 101.


```

student 20 0 222652 3696 3240 R 69.8 0.2 8:45.29 process103
student 20 0 222652 3888 3436 S 21.2 0.2 5:09.59 process102
student 20 0 225952 4088 3504 R 0.2 0.2 0:01.85 top
root 20 0 0 0 0 I 0.2 0.0 0:00.01 kworker/1:3

```

```

[root@serverb ~]# ps jT
PPID    PID    PGID    SID    TTY        TPGID  STAT   UID    TIME  COMMAND
1082    1083   1083    1083   pts/0      3930   Ss     1000   0:00  -bash
1083    1148   1148    1083   pts/0      3930   T      1000   4:16  /bin/bash /ho
1083    1638   1638    1083   pts/0      3930   S      1000   5:15  /bin/bash /ho
1083    2364   2364    1083   pts/0      3930   R      1000   9:01  /bin/bash /ho
1083    2519   2519    1083   pts/0      3930   R      1000   8:01  /bin/bash /ho
1083    3716   3716    1083   pts/0      3930   S      0      0:00  su -
3716    3725   3725    1083   pts/0      3930   S      0      0:00  -bash
1148    3770   1148    1083   pts/0      3930   Z      1000   0:00  [sleep] <defu
1638    3929   1638    1083   pts/0      3930   S      1000   0:00  sleep 1

```

10. Reanude el proceso process101.

-Reanudamos el proceso

```

[root@serverb ~]# pkill -SIGCONT process101

```

11. Finalice process101, process102 y process103 desde la línea de comandos. Verifique que los procesos ya no se muestren en top.

-Finalizamos los procesos.

```

[root@serverb ~]# pkill process101
[root@serverb ~]# pkill process102
[root@serverb ~]# pkill process103
[root@serverb ~]#

```

-Verificamos que ya no se están ejecutando.

```

top - 12:29:43 up 27 min, 1 user, load average: 0.00, 0.00, 0.00
Tasks: 217 total, 1 running, 216 sleeping, 0 stopped, 0 zombie
%Cpu(s): 0.5 us, 0.2 sy, 0.0 ni, 99.3 id, 0.0 wa, 0.1 hi, 0.0 si, 0.0 st
MiB Mem : 5668.6 total, 4039.6 free, 850.8 used, 778.2 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 4549.1 avail Mem

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
2137	student	20	0	4588288	221164	126580	S	1.3	3.8	0:24.41	gnome-shell
2424	student	20	0	1193664	104376	51032	S	0.0	1.8	0:01.37	gnome-software
1428	student	20	0	163512	81868	15828	S	0.0	1.4	0:02.05	jupyter-lab
1985	student	20	0	791816	72872	49976	S	0.3	1.3	0:09.08	Xorg
2394	student	20	0	930904	66192	53472	S	0.0	1.1	0:00.34	evolution-alarm
2432	student	20	0	703016	60036	38372	S	0.0	1.0	0:00.32	seapplet
2531	student	20	0	1062476	54924	44428	S	0.0	0.9	0:00.21	evolution-addre
2191	student	20	0	695556	53104	40824	S	0.0	0.9	0:00.21	gnome-shell-cal
2770	student	20	0	768932	52808	39480	S	0.3	0.9	0:06.55	gnome-terminal-
2305	student	20	0	1145016	52184	41552	S	0.0	0.9	0:00.25	evolution-calen
2209	student	20	0	768288	49232	40684	S	0.0	0.8	0:00.29	evolution-sourc
917	root	20	0	349320	44008	19500	S	0.0	0.8	0:00.68	firewalld
2288	root	20	0	270856	40528	8540	S	0.0	0.7	0:01.78	sssd_kcm
2269	student	20	0	894444	38828	30292	S	0.0	0.7	0:00.09	goa-daemon
2179	student	20	0	604240	36244	24900	S	0.0	0.6	0:01.29	ibus-extension-
2320	student	20	0	956324	35416	26384	S	0.0	0.6	0:00.18	gsd-media-keys

12. Detenga los procesos y regrese a la máquina workstation.

-Salimos de root y detenemos los procesos.

```
[root@serverb ~]# exit
logout
[1]    Terminated                  process101
[2]    Terminated                  process102
[3]-  Terminated                  process103
[student@serverb ~]#
```

-Regresamos al workstation y comprobamos que hemos llevado todo a cabo correctamente.

```
[student@serverb ~]$ exit
logout
Connection to serverb closed.
[student@workstation ~]$ lab grade processes-review

Grading lab.

· Checking lab systems ..... SUCCESS
· Verifying that process101 is not running ..... SUCCESS
· Verifying that process102 is not running ..... SUCCESS
· Verifying that process103 is not running ..... SUCCESS

Overall lab grade: PASS
[student@workstation ~]$
```